Semantic Aware AI Scheduler

Satya Srinivas Paladugu [BL.AI.U4AID24063]
Nandini Padavala [BL.SC.U4CSE24133]
Jahnvi KV [BL.SC.U4ECE24034]

Abstract

When several devices send images to one receiver, their messages may have to wait for the same image decoder. A scheduler decides which message the decoder should process next. This project studies whether information about image importance, mission relevance, repetition and visible scene content can make that decision better. Each complete message contains a small scheduling header and the encoded image data. The scheduler reads the header; the decoder uses the image data to reconstruct the selected image.

The work includes parameter extraction, four-class image-priority classification, a 6000-image dataset, simulated device streams and evaluation of eight rule-based policies and fourteen learned schedulers. On the matched development workload, FIFO recorded an average waiting penalty of 5.8220, SPI-first recorded 3.3741 and LambdaMART recorded 3.2183. Lower penalty means less costly waiting under the declared test rules. LambdaMART was the best of the methods compared, with 44.72 percent lower penalty than FIFO and 4.62 percent lower penalty than SPI-first. These are simulation results using model-generated priority references, rather than verified emergency-response outcomes.

Project goal

The goal is to show important and useful images to the receiver earlier. For example, a routine photograph should not automatically be processed before an image of a serious incident merely because it arrived first. The scheduler must also account for images left waiting. It cannot claim success by ignoring messages it never selects.

Terms used in this report

TABLE 1. Basic terms

Term	Meaning
UE or device	A sender, such as a phone, camera or drone.
Queue	Messages that have arrived and are waiting.
Decoder	The component that reconstructs an image from its encoded data.
Scheduler or policy	The rule or model that chooses the next message.
SPI	The model's estimate of depicted intervention need: Low, Medium, High or Critical.
Waiting budget	A target waiting time used to score the experiment. It is explained with examples in Section 8.

1. The scheduling problem

Imagine that a receiver already has three images waiting. One shows a routine scene, one has High priority and one has Critical priority. A simple first-in-first-out rule processes them in arrival order. A priority rule can choose the Critical image first. Both rules process the same messages. The difference is which information becomes available sooner.

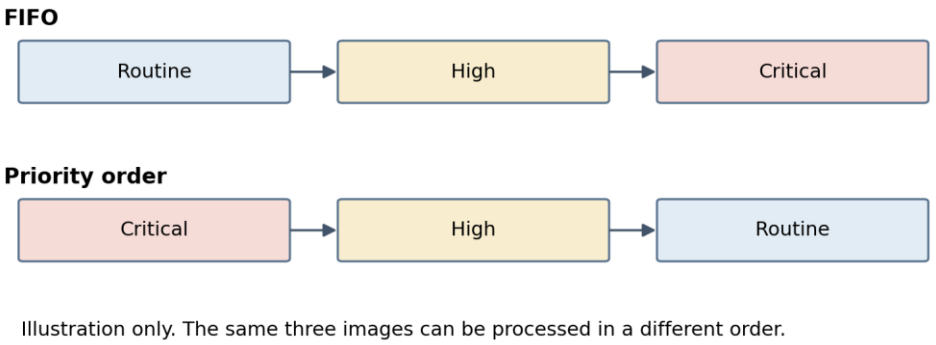


Fig. 1. Illustrative example. FIFO follows arrival order. A priority policy can change the processing order. This drawing is an explanation, not a measured experiment.

What one decision does

Whenever the decoder becomes free, the scheduler looks at all complete messages currently waiting and selects one. Processing then runs to completion. A new message may arrive during that time, but it cannot interrupt the current job. This is called nonpreemptive processing. After the job ends, the scheduler makes a new decision using the updated queue.

Each message receives one attempt. The scheduler does not request a retransmission or retry a poor reconstruction. The current study also does not assign radio channels or control how many bytes a sender transmits. It changes the order of processing after complete messages reach the receiver.

Waiting and processing are different

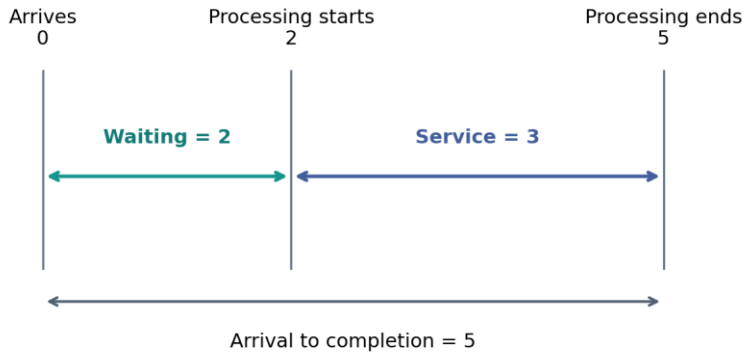


Fig. 2. Illustrative timing example. A message arriving at time 0 and starting at time 2 waited for 2 units. If processing ends at time 5, its service time was 3 and its arrival-to-completion time was 5.

Our main score measures waiting until processing starts. It does not measure the time required to reconstruct the image. In the constant-service experiment, processing one message takes one simulated time unit. The stored scheduler report calls one unit one simulated millisecond. This is a test convention, not a measurement of actual decoder hardware.

2. Header and latent payload architecture

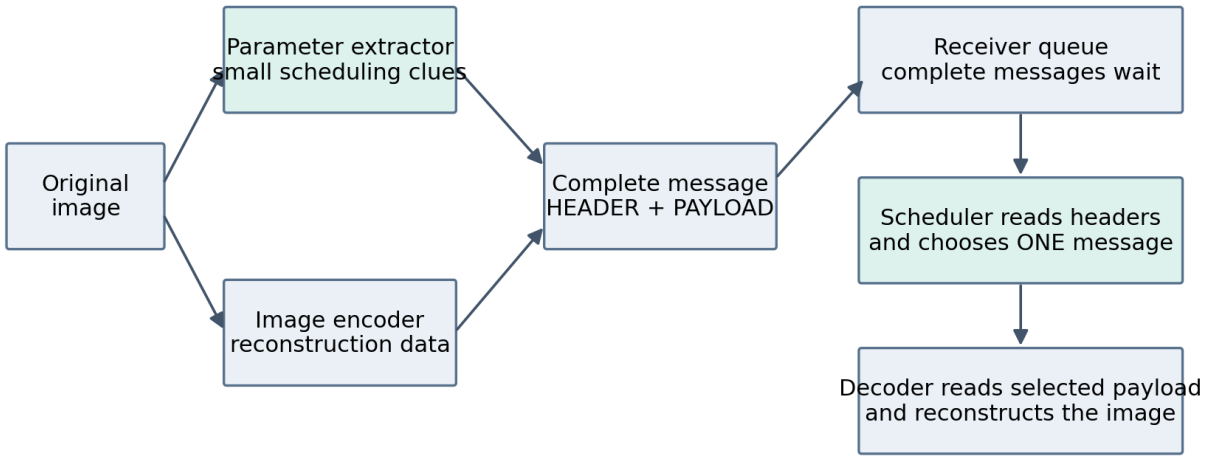


Fig. 3. Two paths create one message. The header contains clues for selection. The payload contains the encoded image representation. The receiver chooses from headers before decoding the selected payload.

What the header contains

The header is the information the scheduler can inspect cheaply. It contains the predicted priority, mission relevance, appearance novelty, scene measurements and the status of those measurements. Identity and timing fields tell the receiver which job it is handling and when it arrived. A status flag distinguishes a missing measurement from a measured zero.

What the latent payload contains

An image encoder transforms the original pixels into a representation that its matching decoder can use. This representation is called a latent. A digital codec may store quantized latent values or codebook indices as bytes. The scheduler treats the complete encoded representation as an opaque payload. It does not need to understand every latent value to choose the next job.

DeepJSCC provides background for learned image transmission [1]. Its original method maps image pixels to complex channel symbols. Those symbols are not automatically an ordinary digital packet. A codec adapter must define the representation, shape, model identity and framing. Our complete-message interface describes the receiver job; it does not establish a finished physical-layer implementation.

Why use parameters instead of decoding everything first

Decoding every waiting image before choosing would spend the very decoder time that the scheduler is trying to manage. Numeric parameters provide a common input across images and codecs. They are still an incomplete description. Two images can have the same object counts but very different meaning, so the scheduler cannot recover facts that the header never captured.

3. Parameters and why each one is needed

TABLE 2. Parameter purpose and literature basis

Parameter	Plain meaning	Reason and paper connection
SPI	How serious does the visible situation appear?	Adds an image-level assessment. CrisisMMD and MEDIC motivate incident-image analysis [5], [6]. Our four urgency levels are project-defined.
Relevance	Does this image match the device's mission?	CLIP compares image and task-text representations [2]. The same image can matter differently for different missions.
Novelty	Does this look different from this sender's recent images?	Separates appearance change from repetition. Content and age are also distinct in semantic freshness work [12]. This is an appearance proxy, not verified new knowledge.
Scene evidence	What configured objects or regions were detected?	YOLO-World and Grounding DINO motivate text-conditioned detection [3], [4]. Counts alone do not establish urgency.
Timing and status	When was it captured or received, and which measurements exist?	Prevents an old image, a failed detector and an empty scene being treated as the same case.

Mission relevance with an example

A drone inspecting road blockage and a camera recording household inventory have different missions. An image of a damaged road may match the first mission strongly and the second weakly. We encode the image and the supported mission description using CLIP, normalize both vectors, and calculate their cosine similarity [2]. A similarity of 0.6 is a model score. It is not a 60 percent probability that the image is useful.

Appearance novelty with an example

The sender compares the current image with its stored images for the same device and mission. If the closest stored image has cosine similarity 0.8, novelty is calculated as one minus 0.8, divided by two, giving 0.1. An identical representation gives zero. The sender queries history before adding the current image, so an image cannot compare against itself.

The executable default keeps 64 recent sender entries unless another capacity is supplied. Empty history has an explicit cold-start flag. Sender history records offered observations, while receiver history records processed outcomes. They serve different purposes. A new viewpoint can look novel without adding a new fact; a small important change can remain visually similar.

4. Scene extraction and representation

What the detector measures

We use YOLO-World with a fixed category list [3]. For each supported category, the extractor records the number of retained detections and the largest bounding rectangle as a fraction of the image. In a 400 by 200 image, a 100 by 80 rectangle occupies 8000 of 80000 pixels, giving an area fraction of 0.1. This is rectangle size in the image, not the object's physical size or distance.

The vocabulary has 93 entries: 80 COCO categories, five additional object categories and eight regions. The added objects are building, bridge, utility pole, traffic sign and animal. The regions are road, water, debris, smoke, flame, snow, ground and grass. Counts apply to 85 object categories. Region-count fields remain unavailable because several water detections do not mean several independent floods.

Why YOLO-World was retained

TABLE 3. Detector-call measurements on the same 300 MEDIC images

Operating point	Median call time	Mean boxes	Zero-detection images
YOLO, confidence 0.25, IoU 0.5, size 640	19.00 ms	1.99	40.0%
Grounding DINO, box 0.35, text 0.25, IoU 0.5	368.69 ms	3.47	13.7%
Grounding DINO, box 0.35, text 0.35, IoU 0.3	369.64 ms	3.52	12.7%

The detector comparison used 16 nouns. It had no reference boxes, so more detections cannot be called better recall. YOLO had lower measured call time and was retained as the lower-cost baseline. These timings do not establish the speed or accuracy of the later 93-class setup. Grounding DINO provides a different text-grounded detection mechanism [4].

Features and bytes are different

The scene input has 374 numbers: 93 count values, 93 area values, 186 availability flags and two detector-status flags. Adding SPI, relevance, novelty and their status fields gives the scheduler a 387-entry base vector. Larger derived queue contexts may contain additional receiver-side values.

The original self-describing scene block occupies 369 bytes. A later compact format uses 307 bytes for dense scenes or 29 plus four times the number of active categories for sparse scenes, under an already installed producer and vocabulary profile. Ten active categories therefore use 69 scene bytes. These figures exclude routing, timing, other semantic fields and the image payload. The old 35-byte scalar header cannot contain this whole scene block.

5. Images and SPI classification

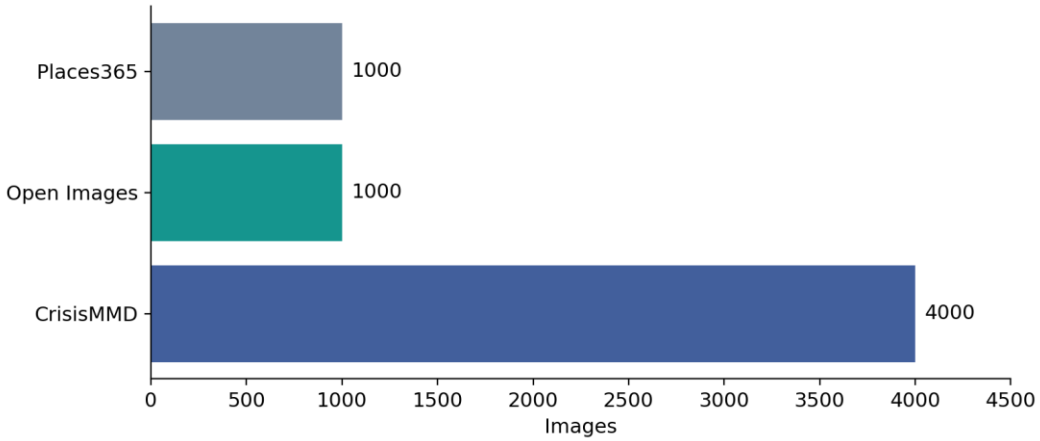


Fig. 4. Image sources in the 6000-image dataset. CrisisMMD supplies the crisis-image labeling cohort; Open Images and Places365 broaden the image pool.

Which images were used and why

The image pool contains 4000 CrisisMMD images, 1000 Open Images images and 1000 Places365 images. The 6000 files contain 5952 distinct raw images. Earlier parameter development used 300 images, including 220 LADI-v2 and 80 MEDIC examples. These stages are separate datasets, not extra independent test images.

CrisisMMD contains disaster-related social-media images and associated humanitarian annotations [5]. MEDIC combines disaster-image sources, including overlapping CrisisMMD material [6]. Such data was suitable for an initial incident-focused SPI task. Its native annotations describe damage, objects or humanitarian categories. They do not directly provide our four intervention-need labels.

Restricting SPI training to crisis imagery limits the claim. A classifier trained there may make errors on ordinary indoor scenes, screenshots or unfamiliar events. Adding everyday images to the receiver dataset broadens inputs but does not prove the classifier generalized correctly to them.

Why four classes

TABLE 4. The four SPI levels

Class	Meaning in this project	Example boundary
Low	No visible problem requiring intervention, or routine content.	An ordinary street scene does not become urgent merely because people are present.
Medium	A visible issue needing attention without clear immediate severe danger.	A maintenance defect can need correction without an immediate emergency.
High	Serious visible damage or disruption supporting prompt attention.	Severe damage does not by itself prove a person is in immediate danger.
Critical	Visible evidence supporting immediate severe intervention need.	The label must not invent hidden victims or an unseen imminent collapse.

Four levels distinguish routine content, nonurgent issues, serious incidents and immediate severe need. Binary incident detection would merge several of these cases. Four levels also keep the classifier output small. The levels are a chosen annotation rubric, not universal emergency categories established by a paper. An unreadable or unsupported image has an unavailable assessment rather than automatically becoming Low.

6. Labeling and training the SPI model

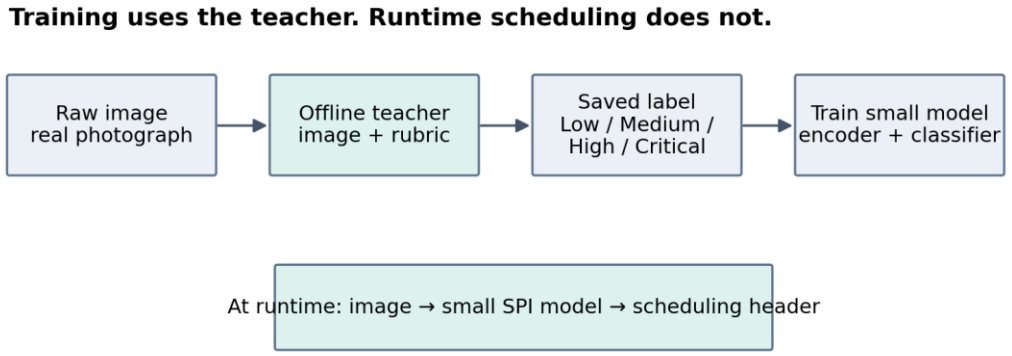


Fig. 5. SPI training and runtime are separate. A VLM supplies offline labels. A smaller image model predicts SPI during parameter extraction.

How labels were obtained

A vision-language model sees an image and a prompt defining visible evidence, the four classes and the required output structure. The system saves the image identity, raw response, parsed label and teacher configuration. Qwen2.5-VL provides the multimodal model foundation [7]. A well-formed response is usable data, but is not proof that the judgment is correct.

The 4000-image labeling file combines 2540 earlier checkpoint assessments and 1460 continuation assessments. The continuation used a pinned Qwen2.5-VL-7B-Instruct checkpoint with thinking disabled. There were 3933 available labels and 67 unavailable assessments. Because the protocols were mixed, this is not described as a single-teacher, single-prompt dataset.

Earlier teacher attempts had malformed responses and unsupported visual claims, including dam or bridge details. Strict parsing detects format failures; it cannot detect every hallucination. The student learns from these AI-generated labels, so its reported accuracy measures agreement with them rather than independently verified real-world urgency. This is supervised classification from pseudo-labels, not automatically reinforcement learning from AI feedback.

TABLE 5. SPI class support after unavailable labels were excluded

Class	Training	Validation	Test	Total
Low	782	419	273	1474
Medium	339	149	200	688
High	899	445	325	1669
Critical	48	24	30	102
Total	2068	1037	828	3933

From an image to a class

The retained SPI pipeline first preprocesses the image, then uses CLIP ViT-B/32 to obtain a 512-value image representation [2]. A logistic-regression head multiplies those values by learned weights and produces four class scores. The head has 2052 scalar coefficients and intercepts. The image encoder accounts for most of the pipeline cost. The CLIP representation used for these predictions is different from the reconstruction codec's latent payload.

7. SPI model comparisons and results

What was compared

The broader study compared four frozen feature encoders: CLIP, SigLIP2, DINO and ConvNeXtV2 [2], [8], [9], [10]. Frozen means their pretrained weights were reused rather than retraining the entire encoder. The saved encoder identity determines which DINO model or fallback was actually used.

The twelve heads were logistic regression, ridge, linear SVM, RBF-SVM, k-nearest neighbors, random forest, extra trees, histogram boosting, MLP, mean-MLP, gated attention MIL and single-query cross-attention. Linear heads test whether the existing features already separate the labels. Nonlinear heads allow more complex boundaries. Token heads examine patch-level features instead of relying only on one pooled vector. Attention-based multiple-instance learning motivates weighted aggregation [11].

The classifier used for the dataset

CLIP with class-balanced logistic regression was fitted on 2068 training examples. Three regularization values, 0.1, 1 and 10, were compared on 1037 validation examples. The selected value was 10. The resulting model was evaluated on the saved 828-image test split and then used to populate SPI predictions in the image dataset.

TABLE 6. SPI classifier results against AI-generated labels

Metric	Result
Validation macro-F1	0.60245
Test macro-F1	0.59997
Test accuracy	64.86%
Low recall	79.49%
Medium recall	36.00%
High recall	69.54%
Critical recall	73.33%

Accuracy counts all correct predictions together. Recall asks how many images of one class were recognized correctly. Macro-F1 gives each class equal influence, so the common classes cannot hide every minority-class error. Medium recall of 36 percent means 72 of its 200 test examples were recognized as Medium.

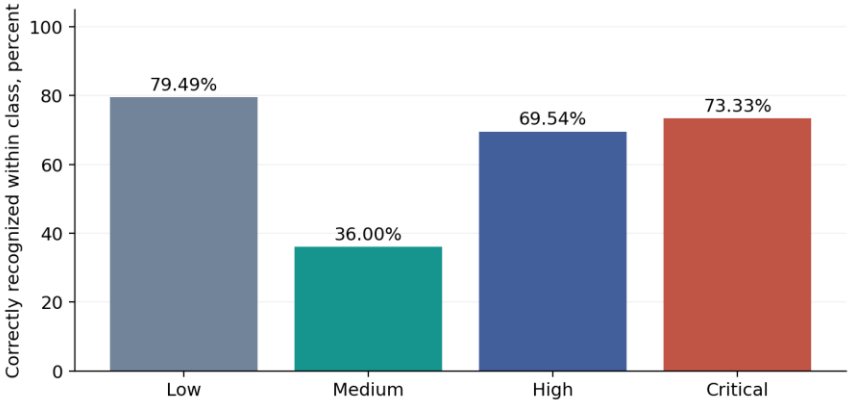


Fig. 6. Class recall for the SPI classifier. Medium was the weakest class. These values measure agreement with pseudo-labels, not human-verified urgency.

8. The waiting budget explained

A budget is a target wait, not a processing delay

In this experiment, a waiting budget means how long a message is allowed to wait before processing should start. It does not force the message to wait that long. A Critical image can start immediately. It also does not mean that its decoder takes two units to run. Every job takes one unit in the constant-service control, regardless of class.

TABLE 7. Chosen waiting targets for the scheduling experiment

Reference class	Target waiting time	Meaning
Critical	2 simulated units	Waiting becomes late after 2 units.
High	4 simulated units	Waiting becomes late after 4 units.
Medium	8 simulated units	Waiting becomes late after 8 units.
Low	16 simulated units	Waiting becomes late after 16 units.

The numbers 2, 4, 8 and 16 were chosen as experimental settings to make postponing important images more costly. They are not emergency-service standards and were not discovered by the VLM. Using them lets every policy face the same scoring rule. Missing urgency references cannot be filled by simply assigning Low.

Why waiting creates a penalty

A short wait is usually less serious than a long wait. The same wait also matters more when the image has a shorter target. First divide the actual wait by the target. After the target is exceeded, add the excess fraction again. The result is called a waiting penalty. Smaller is better.

For a wait w and a target b, penalty = w/b + max of zero and w/b minus one. Before the target, the penalty rises steadily. After the target, it rises twice as fast. At exactly the target, the penalty is 1.

TABLE 8. Worked examples for the same scoring rule

Image	Wait	Calculation	Penalty
Critical, target 2	1	1/2 + 0	0.50
Critical, target 2	2	2/2 + 0	1.00
Critical, target 2	4	4/2 + (4/2 − 1)	3.00
Low, target 16	4	4/16 + 0	0.25

Thus a four-unit wait costs 3 for a Critical image and 0.25 for a Low image. This is how the test expresses that delaying the Critical image is more costly. The penalty is a score with no time unit. A score of 3 does not mean three milliseconds.

9. How a complete schedule receives its score

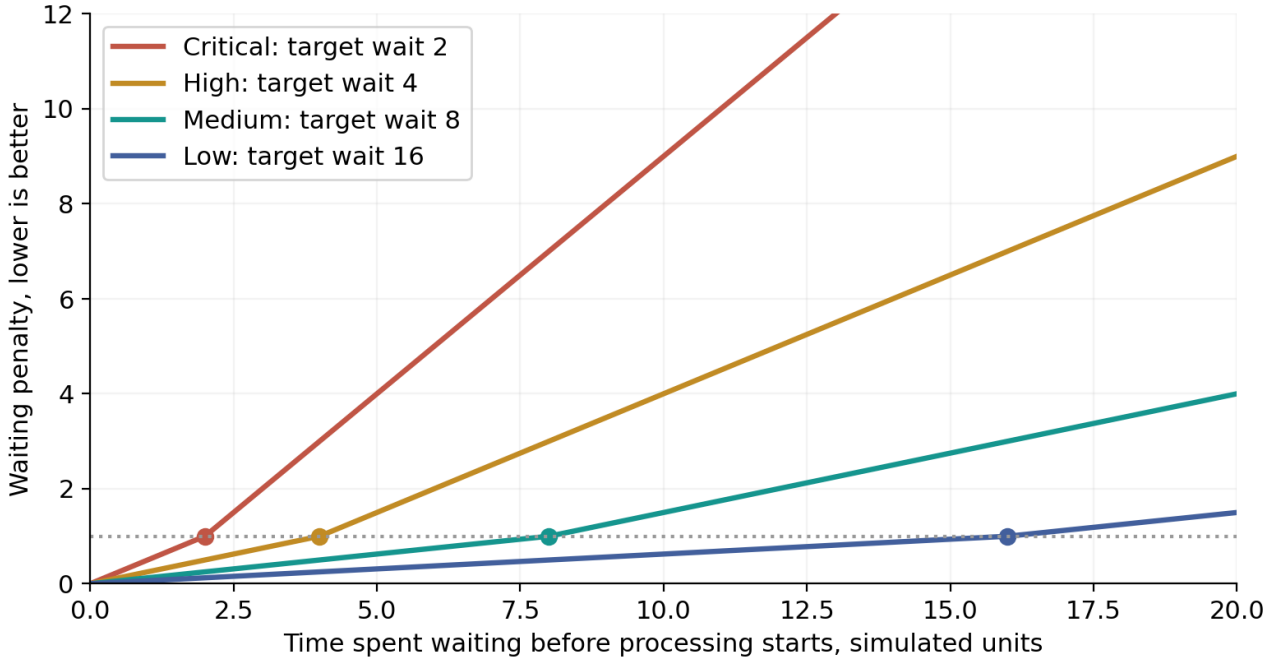


Fig. 7. The waiting-penalty curves for the chosen targets. Each dot marks the target wait, where penalty equals 1. Critical waiting accumulates penalty fastest.

A three-image example

Suppose Routine, High and Critical images all become ready at time 0, and each takes one unit to process. Assume the Routine image arrived first among that tied group. FIFO processes Routine, then High, then Critical. Their waits are 0, 1 and 2. Their penalties are 0, 0.25 and 1, giving a mean of 0.4167.

SPI-first processes Critical, then High, then Routine. Their waits are 0, 1 and 2, but the penalties are now 0, 0.25 and 0.125. The mean is 0.125. Both policies processed three images in three units. SPI-first received a better score because it placed the costly wait on the less urgent image. This example assumes correct priority labels and is illustrative, not a measured run.

Images left in the queue still count

The experiment adds the penalty of every assessable offered image and divides by the number of those images. An image that starts processing stops accumulating waiting penalty. Its existing penalty remains in the total. An image still waiting when the episode ends contributes its elapsed wait up to that endpoint. Otherwise a policy could appear good by serving only a few easy images and ignoring the rest.

How the score is used for learning

Supervised schedulers learn which candidate would cause the most extra penalty if it waited one more service unit. Reinforcement-learning agents receive a negative reward when total waiting penalty increases. The stored reward divides that increase by 100 to control its numeric scale. This scaling is a training setting, not another waiting target.

Reference priority is kept separate from runtime inputs. A policy acts on predicted SPI and permitted context; the evaluator scores it using the saved reference. The current references are silver labels, meaning model-generated judgments rather than independently verified operational urgency.

10. Traditional and fixed-rule scheduling algorithms

These methods do not train a queue-selection model. Each follows a fixed rule. Some ignore image meaning, while others use the extracted semantic parameters. All choose from the same ready queue and obey the same one-job-at-a-time rule.

TABLE 9. The eight fixed-rule policies evaluated

Policy	How it chooses	What it tests
FIFO	Choose the earliest arrival.	Whether arrival order alone is sufficient.
Round robin	Rotate service among senders.	Fairness between devices.
Random	Choose a random ready message using a saved seed.	A chance-based control.
SPI-first	Choose the highest predicted priority; break ties by arrival.	A simple importance-aware rule.
Relevance-first	Choose the strongest mission match.	Whether task alignment alone gives a useful order.
Novelty-first	Choose the largest appearance change.	Whether visual difference alone gives a useful order.
SPI then novelty	Choose priority first and novelty within the class.	Whether repetition helps resolve priority ties.
Predicted EDF	Choose the earliest arrival-plus-predicted-budget deadline.	A time-aware rule based on predicted classes.

Why these are useful comparisons

FIFO and round robin answer familiar questions: should the first arrival go first, or should devices take turns? SPI-first answers the stronger question: can a simple priority rule already solve much of this test? A learned method should be compared with that rule as well as FIFO. Otherwise its advantage may mostly come from using priority information at all.

Novelty-first can select a visually different but unimportant scene. Relevance-first can favor a mission match without recognizing immediate need. Predicted EDF uses an experimental deadline derived from a predicted class, not a real application deadline. These differences explain what each rule measures without assuming in advance which one must win.

11. Learned scheduling algorithms

A learned scheduler receives numeric inputs and produces a score for each waiting candidate. The receiver selects the candidate with the highest score. Different models learn that score in different ways. The architecture describes how the score is computed; the loss or reward describes how training changes it.

TABLE 10. Supervised methods and their plain-language roles

Method	How it learns or scores
Linear choice	Adds learned weighted contributions from the inputs. It is the simplest trained scorer.
MLP choice	Uses a small neural network to learn nonlinear combinations and select a candidate.
Delay regression	Predicts the extra waiting cost caused by postponement. Huber loss reduces the influence of large individual errors.
Priority classifier	Predicts four reference classes and uses them for waiting-sensitive selection.
RankNet	Learns which member of a candidate pair should be ahead [13].
LambdaMART	Combines many small decision trees to learn useful ordering [14].
Deep Sets RankNet	Scores a candidate using its own fields plus pooled queue and history information [15].
Tie-aware listwise	Learns an ordering across several candidates while respecting equal-cost cases. The implementation adapts listwise learning [17].
Attention set ranker	Lets candidate representations exchange information before ranking. It is an adaptation of set attention [16].
Temporal Deep Sets	Adds a GRU that reads up to three recent observations in order before scoring.

What a pairwise ranker learns

If waiting one more unit is more costly for image A than image B, a pairwise ranker should give A the higher score. RankNet turns the score difference into a pair preference and adjusts its weights when the preference is wrong [13]. It learns ordering rather than a reconstruction of the image.

What history adds

A set model summarizes the current queue without treating array position as meaning. A temporal model also examines the order of recent observations. These are possible ways to use context. Their extra complexity does not guarantee better results, so they are compared on the same recorded workload.

12. Reinforcement learning and training limits

TABLE 11. The four reinforcement-learning methods

Method	Plain-language explanation
PPO	Learns a probability for each legal choice and adjusts the policy using accumulated reward [20].
Double DQN	Learns the value of choosing a candidate. Separate networks choose and evaluate the next action during training [18].
Double DQN with prioritized replay	Revisits stored training transitions more often when their prediction errors are larger [19]. This replay priority is not operational image priority.
Ranker-initialized PPO	Starts PPO from an already trained ranker instead of a fresh actor, then updates it using queue rewards.

Legal-action masks prevent the agent from selecting a message that is not ready. A decision advances through service and any required idle interval to the next decision point. The environment charges waiting accumulated during those intervals. New arrivals are therefore relevant even when the server is busy.

Training budget is different from waiting budget

A waiting budget tells the evaluator when an image is late. A training budget tells us when to stop running a learning experiment. For example, 20000 action decisions means an RL learner was allowed that many choices during training. It does not mean that an image may wait for 20000 units.

TABLE 12. Recorded scheduler training settings

Item	Setting
Supervised training	64 development episodes and 512 sampled decision snapshots per method.
Supervised epoch limit	Up to 50 passes through training data, with early stopping after eight validation checks without improvement.
RL interaction limit	20000 action decisions per method.
Model seed	1729.
Validation workload	15 traces spanning device counts and offered loads.
Paired notebook runtime	Python 3.13.15, Torch 2.11.0, two T4 GPUs per paired notebook.
Recorded paired run durations	About 2 hours 56 minutes and 2 hours 46 minutes, including their assigned execution work.

The project has struggled to complete its intended model comparison and repeated evaluation within available compute, GPU memory, storage and runtime limits. Fourteen scheduler methods nevertheless completed development evaluation. These limits do not establish that they converged. Some RL checkpoints became worse later in training. The full SPI comparison also stopped after disk exhaustion, so its unfinished parts cannot be described as completed.

13. How the scheduling comparison was run

A shared workload for a fair comparison

The simulator saves message arrivals, processing times and possible outcomes before comparing policies. Each policy receives the same realization. Thus a policy is not allowed a different, easier set of arrivals just because it is a different algorithm. Only complete messages become eligible, and later arrivals remain unavailable until their arrival time.

Development validation includes device populations of 4, 12, 50, 60 and 100. Fifteen mission templates represent phones, inspection and search drones, vehicle cameras, CCTV, industrial robots and other sources. A device mission supplies context, not an automatic priority. A normal phone can capture a serious incident; an inspection drone can send a routine repeat.

Population and traffic are different

One test keeps total offered load fixed while changing the number of devices. Another keeps each device's rate fixed, so adding devices also adds traffic. Bursts concentrate arrivals in time. Joining and pausing devices alter achieved traffic. These controls separate the effect of more senders from the effect of more messages.

What the reported cohort covers

The broad pool contains 6000 images. The saved receiver report lists 5376 training, 417 validation and 207 test records for that pool, with a separate 3915-record cohort having assessable delay references. Those counts differ from the SPI classifier's 2068/1037/828 split because the experiments use different eligible records and roles. They must not be presented as one split.

The following comparison uses saved development results, not the protected final scheduler test. Model-generated priority references define the assessable urgency scope. Simulated times and mixed image streams are not live recordings of independent emergency incidents. The reported numeric differences describe these runs; they are not confidence intervals or evidence of deployment safety.

Reading the next table

All methods use the same waiting-penalty definition. Smaller is better. The percentage improvement over FIFO is the FIFO penalty minus the method penalty, divided by the FIFO penalty, then multiplied by 100. A negative value means the method was worse than FIFO. The colors in the plot distinguish ordinary scheduling, fixed semantic rules and learned policies.

14. Direct comparison of all evaluated algorithms

TABLE 13. All eight fixed rules and fourteen learned schedulers on the development comparison

Algorithm	Type	Penalty ↓	Change vs FIFO
LambdaMART	Learned	3.2183	+44.72%
Delay regression	Learned	3.3573	+42.33%
RankNet	Learned	3.3642	+42.22%
SPI-first	Semantic rule	3.3741	+42.05%
Deep Sets RankNet	Learned	3.3819	+41.91%
Tie-aware listwise	Learned	3.3859	+41.84%
SPI then novelty	Semantic rule	3.4349	+41.00%
Temporal Deep Sets	Learned	3.4480	+40.78%
Attention set ranker	Learned	3.4520	+40.71%
MLP choice	Learned	3.5675	+38.72%
Ranker + PPO	Learned	3.6375	+37.52%
Priority classifier	Learned	3.8271	+34.26%
Linear choice	Learned	4.3392	+25.47%
PPO	Learned	4.5935	+21.10%
Double DQN	Learned	4.7099	+19.10%
Relevance-first	Semantic rule	4.9203	+15.49%
Predicted EDF	Semantic rule	5.0266	+13.66%
Double DQN + replay priority	Learned	5.1757	+11.10%
Round robin	Traditional	5.8059	+0.28%
FIFO	Traditional	5.8220	+0.00%
Random	Traditional	5.9196	-1.68%
Novelty-first	Semantic rule	6.1917	-6.35%

LambdaMART recorded the lowest penalty, 3.2183. FIFO recorded 5.8220 and round robin recorded 5.8059. SPI-first recorded 3.3741. Thus the best learned model improved on FIFO by 44.72 percent, while the fixed SPI-first rule improved on FIFO by 42.05 percent. The additional reduction from SPI-first to LambdaMART was 4.62 percent. These percentages describe waiting penalty, not time saved in milliseconds.

15. Comparison plots

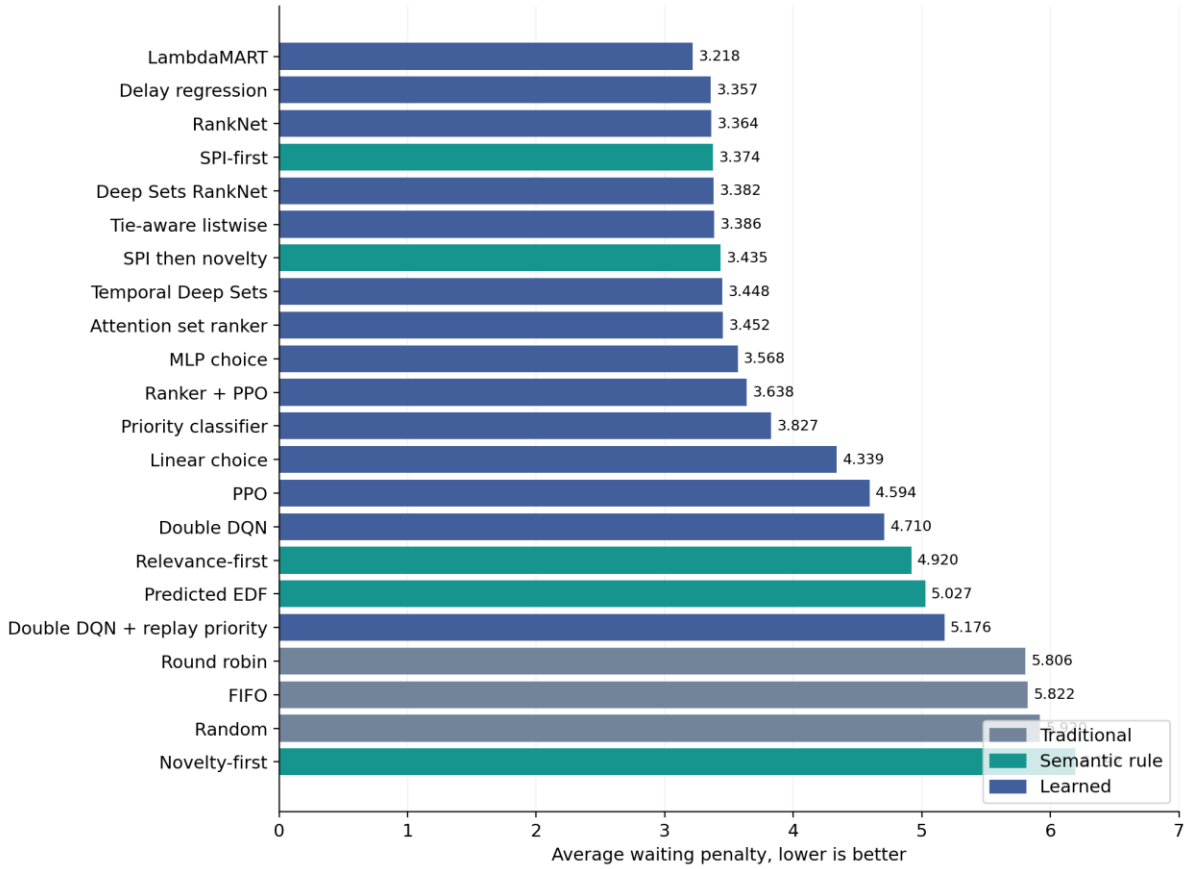


Fig. 8. All 22 evaluated methods, sorted from lowest to highest penalty. Ordinary policies are gray, fixed semantic rules are green and learned methods are blue. The same scoring rule is used throughout.

The plot makes the main comparison visible. Most learned models beat FIFO in this run, but a learned model is not automatically better than SPI-first. Delay regression and RankNet joined LambdaMART in numerically beating SPI-first. The remaining eleven learned methods did not. Novelty-first was worse than FIFO, showing that visual difference alone did not match this waiting objective well.

16. What changes when priority is used

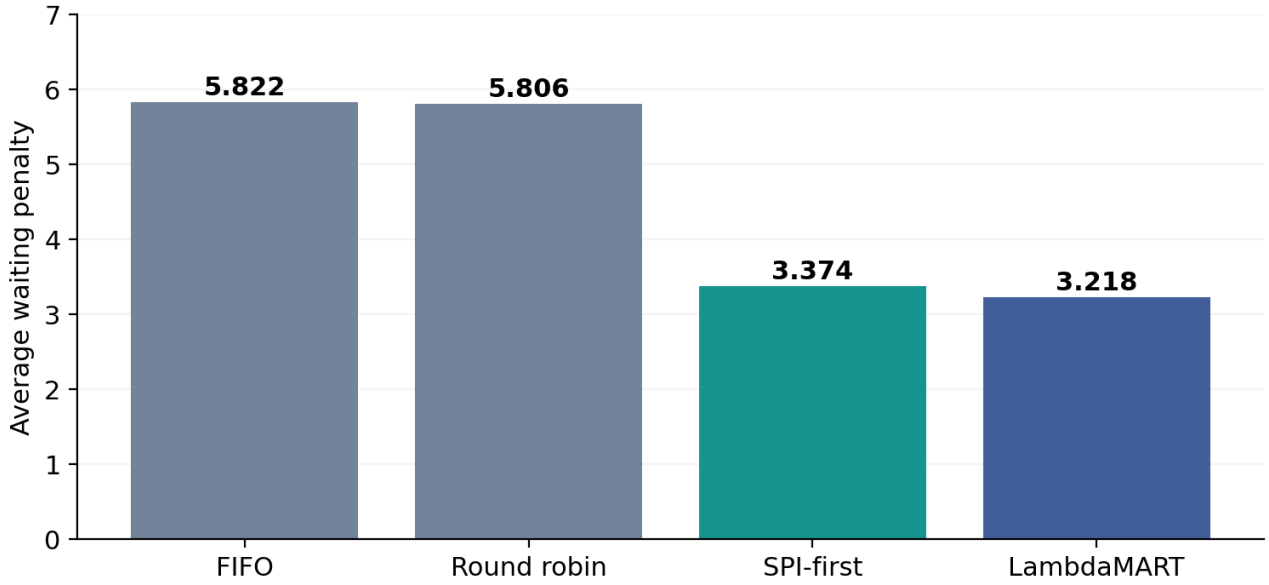


Fig. 9. Four representative methods on the same development comparison. Most of the observed FIFO-to-LambdaMART reduction was already achieved by the simple SPI-first rule.

Traditional policies

FIFO and round robin produced similar aggregate penalties. Round robin shares opportunities among senders, but it does not automatically recognize an important image. Random was slightly worse than FIFO. These results show the limitation of ignoring image-priority information under a priority-sensitive scoring rule.

Fixed semantic rules

SPI-first was the strongest fixed semantic rule in this development table. SPI then novelty was close but had a higher penalty, 3.4349. Relevance-first and predicted EDF improved on FIFO but did not match SPI-first. Novelty-first had penalty 6.1917, which was worse than FIFO. A surprising image is not necessarily a costly image to postpone.

Learned models

LambdaMART, delay regression and RankNet had the three lowest learned penalties. The temporal and attention models did not improve on the simpler SPI-first control in this run. The four RL methods also had higher penalties than SPI-first; ranker-initialized PPO was the best among those four. This is the outcome of the recorded training settings, not a universal ranking of algorithm families.

The target explains part of this result. It charges waiting according to reference priority and elapsed wait. It does not independently reward every visual novelty or mission distinction within a priority class. The comparison therefore measures success under that declared waiting rule, rather than complete semantic understanding.

17. Which classes waited less

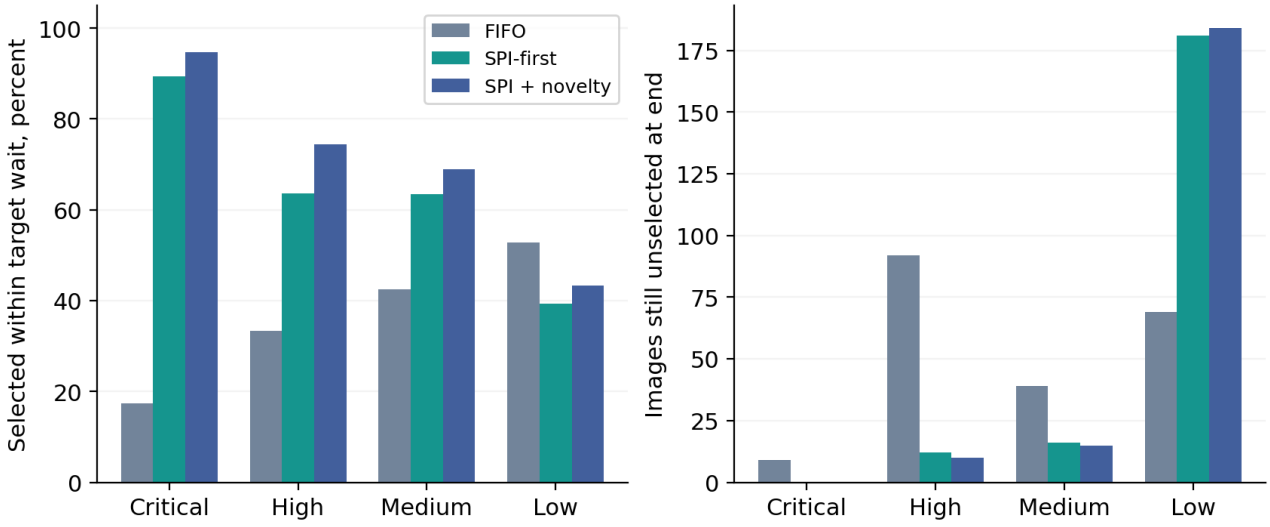


Fig. 10. Left: the percentage selected within the class target. Right: messages still unselected when the episode ended. Both views matter because improving urgent service can increase waiting elsewhere.

TABLE 14. FIFO and SPI-first on the same 15 validation episodes

Policy	Class	Offered	Unselected	Selected mean wait	Within target
FIFO	Critical	75	9	21.441	17.3%
SPI-first	Critical	75	0	2.929	89.3%
FIFO	High	1087	92	22.099	33.3%
SPI-first	High	1087	12	8.314	63.6%
FIFO	Medium	435	39	20.884	42.5%
SPI-first	Medium	435	16	15.005	63.4%
FIFO	Low	846	69	19.282	52.8%
SPI-first	Low	846	181	34.582	39.4%

For Critical images, SPI-first left none unselected and had a selected mean wait of 2.929 simulated units, compared with FIFO's 21.441. However, it left 181 Low images unselected, compared with FIFO's 69. Prioritization therefore changed who received service sooner; it did not make every class better at the same time.

Selected mean wait includes only messages that started processing. It must be read with unselected counts. The within-target percentage uses the evaluator's eligible follow-up observations. Those denominators differ, so a rounded percentage should not be reconstructed simply by dividing selected messages by all offered messages. Class-level learned-model outcomes are not supplied by the retrieved aggregate table, so the report does not assign LambdaMART an invented Critical-class benefit.

18. Traffic-load comparison

TABLE 15. Fixed-rule results over 80 paired workload traces

Policy vs FIFO	Lower penalty	Equal penalty	Higher penalty	Mean penalty difference
Round robin	32	5	43	−0.02059
SPI-first	74	6	0	−1.70285
SPI then novelty	74	6	0	−1.62261

This separate rate study contains 80 traffic traces and four policies, giving 320 policy results. Each row above compares a policy with FIFO on identical traces. A negative difference means less waiting penalty than FIFO. SPI-first and SPI then novelty were lower on 74 traces and tied on six. Round robin varied more, winning on 32 and losing on 43.

The rate study varied device count, fixed-total versus per-device traffic, burst level and sampling mode. It completed in about 44 seconds on CPU. This table is a paired workload comparison, not the same aggregate as the earlier 15-trace learned-model validation. The two studies are therefore shown separately.

Why load affects the result

When messages arrive slowly enough, the queue may often be empty. Most methods can start a message immediately and there is little order to improve. When arrivals form a backlog, the scheduler must choose who waits. Under overload, some images can still be waiting when the observation window ends. A lower priority-weighted score does not mean that the server processed more images per second.

Observed RL training behavior

TABLE 16. Best observed and final RL checkpoints

Method	Best observed decisions	Best penalty	Final penalty
PPO	20000	4.5935	4.5935
Double DQN	10000	4.7099	5.4125
Double DQN + replay priority	16000	5.1757	5.3716
Ranker + PPO	2048	3.6375	4.1661

For three of these methods, a later checkpoint was worse than the best saved one. More training choices did not guarantee steady improvement. The reported scheduler comparison uses the selected development checkpoint. These curves do not establish convergence or stability across independent seeds.

19. Computational cost and experiment scope

Where resources were spent

The expensive stages were not limited to scheduler scoring. They included VLM labeling, image-encoder feature extraction, storage of patch-level features, classifier searches and RL interactions. A 49 by 768 CLIP token array uses 150528 bytes per image in float32. Storing it for 3933 images takes about 565 MiB before compression and metadata. Multiple feature banks and checkpoints multiply that cost.

The broader SPI comparison completed twelve original-setting heads each for CLIP, SigLIP2 and DINO, for 36 completed candidate results. ConvNeXtV2 feature extraction stopped when storage filled after about 97 minutes. Other balancing regimes and the complete final selection did not finish. The best development candidate among the 36 was CLIP with single-query cross-attention, macro-F1 0.63365. This is a different candidate and evaluation role from the dataset classifier's test macro-F1 0.59997.

Measured SPI inference cost

TABLE 17. Raw-image-to-SPI profile of CLIP plus logistic regression

Measurement	CPU	GPU
Median call	104.74 ms	28.56 ms
95th percentile	112.46 ms	38.84 ms
Timed calls	30	100

The profile used the same ten development images, ten warmups, three CPU repeats per image and ten synchronized GPU repeats. It covers the SPI path, not the full detector, mission, novelty and serialization pipeline. The older detector timings used another scope, so their percentile values cannot simply be added to these values to create a measured full-header percentile.

What the evidence supports

The experiments support a completed extraction dataset, measured pseudo-label agreement for SPI, and development comparisons of fixed and learned schedulers. They do not establish general emergency understanding, exact image recovery or measured end-to-end radio latency. The SPI test cohort had already been exposed during development. The protected scheduler final test remained unopened in the supplied records. The delay references are model-generated, and queue variations from one seed are not independent disaster events.

Conclusion

Semantic priority changed processing order enough to reduce the recorded waiting penalty substantially compared with FIFO. LambdaMART gave the lowest score among the 22 compared methods, but SPI-first captured most of the observed improvement with a fixed rule. The results therefore distinguish two findings: using priority information helped strongly in this experiment, while the extra gain from learning was smaller. The report states both results together, with the waiting rule, workload and resource limits that define them.

Appendix A. Complete SPI encoder comparison

The values below are development macro-F1, not final test scores. Macro-F1 averages the four class F1 scores equally. All 36 rows completed in the original-setting comparison; ConvNeXtV2 did not complete its feature extraction. The same numbers are used in the source completed-comparison CSV.

TABLE 18. All completed encoder and classifier pairs

Classifier head	CLIP	SigLIP2	DINO
logistic	0.4398	0.4401	0.5317
ridge	0.4517	0.4624	0.5249
linear_svm	0.4862	0.5490	0.5783
rbf_svm	0.6141	0.6003	0.5796
knn	0.5832	0.6031	0.5686
random_forest	0.4193	0.4250	0.4130
extra_trees	0.4386	0.4424	0.4364
hist_boost	0.5386	0.5320	0.5092
mlp	0.5896	0.6152	0.5835
mean_mlp	0.6120	0.6031	0.5592
gated_abmil	0.6079	0.5717	0.5661
single_query_cross_attention	0.6337	0.6263	0.5710

Logistic and ridge heads use linear decision boundaries. SVM adds either a linear or nonlinear boundary. k-nearest neighbors compares a new vector with stored training vectors. Tree ensembles combine many split-based rules. MLP learns nonlinear combinations. Mean-MLP averages patch features, gated attention MIL learns their weights, and single-query cross-attention learns a query that collects useful patch information. Each head must use the same encoder and preprocessing on which it was fitted.

References

[1] Bourtsoulatze, Kurka and Gunduz. Deep Joint Source-Channel Coding for Wireless Image Transmission. 2019. https://arxiv.org/abs/1809.01733

[2] Radford et al. Learning Transferable Visual Models From Natural Language Supervision. ICML 2021. https://proceedings.mlr.press/v139/radford21a.html

[3] Cheng et al. YOLO-World Real-Time Open-Vocabulary Object Detection. CVPR 2024. https://arxiv.org/abs/2401.17270

[4] Liu et al. Grounding DINO. 2023, revised 2024. https://arxiv.org/abs/2303.05499

[5] Alam, Ofli and Imran. CrisisMMD Multimodal Twitter Datasets from Natural Disasters. ICWSM 2018. https://arxiv.org/abs/1805.00713

[6] MEDIC dataset and documented annotation tasks. https://crisisnlp.qcri.org/medic/

[7] Bai et al. Qwen2.5-VL Technical Report. 2025. https://arxiv.org/abs/2502.13923

[8] Tschannen et al. SigLIP 2. 2025. https://arxiv.org/abs/2502.14786

[9] Oquab et al. DINOv2 Learning Robust Visual Features without Supervision. 2023. https://arxiv.org/abs/2304.07193

[10] Woo et al. ConvNeXt V2. CVPR 2023. https://arxiv.org/abs/2301.00808

[11] Ilse, Tomczak and Welling. Attention-based Deep Multiple Instance Learning. ICML 2018. https://proceedings.mlr.press/v80/ilse18a.html

[12] Maatouk, Assaad and Ephremides. The Age of Incorrect Information. 2020, revised 2022. https://arxiv.org/abs/2012.13214

[13] Burges et al. Learning to Rank using Gradient Descent. ICML 2005. https://www.microsoft.com/en-us/research/publication/learning-to-rank-using-gradient-descent/

[14] Burges. From RankNet to LambdaRank to LambdaMART An Overview. 2010. https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/

[15] Zaheer et al. Deep Sets. NeurIPS 2017. https://arxiv.org/abs/1703.06114

[16] Lee et al. Set Transformer. ICML 2019. https://proceedings.mlr.press/v97/lee19d.html

[17] Xia et al. Listwise Approach to Learning to Rank Theory and Algorithm. ICML 2008. https://www.microsoft.com/en-us/research/publication/listwise-approach-to-learning-to-rank-theory-and-algorithm/

[18] van Hasselt, Guez and Silver. Deep Reinforcement Learning with Double Q-learning. AAAI 2016. https://arxiv.org/abs/1509.06461

[19] Schaul et al. Prioritized Experience Replay. ICLR 2016. https://arxiv.org/abs/1511.05952

[20] Schulman et al. Proximal Policy Optimization Algorithms. 2017. https://arxiv.org/abs/1707.06347